A Deep Dive Introduction to Next.js

Easy Guide

Contents

Part I

Next.js App Router (Modern Approach)

Chapter 1

Creating a Next.js Project

1.1 What is Next.js?

Next.js is a React framework that makes building web applications easier and faster. It adds powerful features on top of React.

1.1.1 Why Next.js?

- File-based routing (no config needed)
- Server-side rendering for fast apps
- Static generation for performance
- API routes built-in
- Automatic code splitting
- Image optimization
- Easy deployment

1.2 Installation

```
npx create-next-app@latest my-app
cd my-app
npm run dev
```

The app starts at `http://localhost:3000`

1.3 Project Structure

```
my-app/
  app/                    # App Router directory
     layout.js           # Root layout
     page.js             # Home page (/)
     dashboard/
```

```
        page.js        # /dashboard page
  public/              # Static files
  node_modules/
  package.json
  next.config.js       # Next.js configuration
```

1.4 First Page

Open `app/page.js`:

```
export default function Home() {
  return (
    <main>
      <h1>Welcome to Next.js!</h1>
      <p>Your app is ready</p>
    </main>
  );
}
```

This is automatically the home page at `/`
Magic: No routes config needed. Files = Routes!

Chapter 2

Understanding File-based Routing & React Server Components

2.1 File-based Routing

Files in the `app/` directory automatically become routes.

```
app/page.js                → /
app/about/page.js          → /about
app/blog/page.js           → /blog
app/products/[id]/page.js  → /products/123 (dynamic)
```

> No configuration needed!

2.2 React Server Components

By default, Next.js components run on the server.

2.2.1 Advantages

- Direct database access
- No API needed
- Sensitive data stays hidden
- Smaller JavaScript bundle
- Faster page load

2.2.2 Server Component

```
// app/posts/page.js
export default function Posts() {
  // This runs on server only!
  const posts = fetchPostsFromDatabase();
```

```
  return (
    <div>
      {posts.map(post => (
        <div key={post.id}>{post.title}</div>
      ))}
    </div>
  );
}
```

2.3 When to Use Server vs Client

Need	Server Component	Client Component
Fetch data	Yes	No
Database access	Yes	No
API keys	Yes	No
Interactivity	No	Yes
useState/hooks	No	Yes

Default: Use server components, add client only when needed!

Chapter 3

Adding Routes via the Filesystem

3.1 Basic Routes

Create files in `app/` directory:

```
// Create app/about/page.js
export default function About() {
  return <h1>About Us</h1>;
}

// Now /about works!
```

3.2 Nested Routes

```
app/
  blog/
     page.js          → /blog
     [slug]/
        page.js       → /blog/my-post
     archive/
        page.js       → /blog/archive
```

3.3 Adding Multiple Routes

```
app/products/page.js       → /products
app/products/new/page.js   → /products/new
app/products/[id]/page.js  → /products/123
app/cart/page.js           → /cart
app/account/page.js        → /account
```

Pattern: Each `page.js` = one route!

Chapter 4

Navigating Between Pages

4.1 Link Component

Navigate without page reload:

```
import Link from 'next/link';

export default function Home() {
  return (
    <nav>
      <Link href="/">Home</Link>
      <Link href="/about">About</Link>
      <Link href="/blog">Blog</Link>
    </nav>
  );
}
```

4.2 Programmatic Navigation

```
'use client';

import { useRouter } from 'next/navigation';

export default function Button() {
  const router = useRouter();

  const handleClick = () => {
    router.push('/dashboard');
  };

  return <button onClick={handleClick}>Go to Dashboard</button>;
}
```

Note: `'use client'` needed for interactivity.

4.3 Dynamic Routes

```
// Link to dynamic route
<Link href={`/products/${productId}`}>
  {productName}
</Link>
```

Pattern: Link for navigation, useRouter for buttons!

Chapter 5

Working with Pages & Layouts

5.1 Layout Component

Wrap multiple pages with shared layout:

```
// app/layout.js (Root layout)
export const metadata = {
  title: 'My App',
  description: 'Generated by create next app'
};

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <header>
          <nav>Navigation</nav>
        </header>
        {children}
        <footer>Footer</footer>
      </body>
    </html>
  );
}
```

5.2 Nested Layouts

```
// app/blog/layout.js
export default function BlogLayout({ children }) {
  return (
    <div>
      <aside>Blog Navigation</aside>
      <main>{children}</main>
    </div>
  );
```

```
}

// All pages in /blog get this layout
// app/blog/page.js
// app/blog/[slug]/page.js
```

5.3 Layout Hierarchy

```
Root Layout (app/layout.js)
   Blog Layout (app/blog/layout.js)
       Blog Page (app/blog/page.js)
       Post Page (app/blog/[slug]/page.js)
   Products Layout (app/products/layout.js)
        Products Page (app/products/page.js)
        Product Detail (app/products/[id]/page.js)
```

Pattern: Layouts wrap pages at same folder level and below!

Chapter 6

Reserved File Names & Custom Components

6.1 Reserved Filenames

File	Purpose
`page.js`	Route content
`layout.js`	Shared layout
`loading.js`	Loading state (Suspense)
`error.js`	Error boundary
`not-found.js`	404 page
`route.js`	API endpoint

6.2 Custom Components

Store reusable components elsewhere:

```
app/
  components/
     Header.js
     Footer.js
     Button.js
  layout.js
  page.js
```

6.3 Organizing a Next.js Project

Best practices:

```
app/                    # Routes
  (auth)/              # Grouped routes (parentheses hide in URL)
     login/
     signup/
  (dashboard)/
     layout.js
```

```
      page.js
   components/         # Shared components
   lib/                # Utilities and helpers
   styles/             # Global styles
   public/             # Static files

public/
   logo.png
   images/

.env.local              # Secrets (never commit)
next.config.js          # Next.js configuration
package.json
```

Tip: Use parentheses for route groups!

Chapter 7

Configuring Dynamic Routes & Using Route Parameters

7.1 Dynamic Route Syntax

Use square brackets for parameters:

```
app/posts/[id]/page.js    → /posts/123
app/users/[userId]/page.js → /users/john
```

7.2 Accessing Parameters

```
// app/posts/[id]/page.js
export default function PostPage({ params }) {
  const { id } = params;

  return (
    <div>
      <h1>Post {id}</h1>
      {/* Fetch post using id */}
    </div>
  );
}
```

7.3 Multiple Dynamic Segments

```
// app/users/[userId]/posts/[postId]/page.js
export default function UserPost({ params }) {
  const { userId, postId } = params;

  return (
    <div>
      <h1>User {userId}, Post {postId}</h1>
    </div>
  );
```

```
}

// /users/john/posts/123 → { userId: 'john', postId: '123' }
```

7.4 Catch-all Routes

Capture all remaining segments:

```
// app/docs/[...slug]/page.js
export default function Docs({ params }) {
  const { slug } = params;  // Array of segments

  console.log(slug);
  // /docs/guide/getting-started → ['guide', 'getting-started']
}
```

Pattern: [id] for one, [...slug] for multiple!

Chapter 8

Adding a Custom Component To A Layout

8.1 Layout with Components

```
// app/layout.js
import Header from '@/components/Header';
import Footer from '@/components/Footer';

export default function RootLayout({ children }) {
  return (
    <html>
      <body>
        <Header />
        {children}
        <Footer />
      </body>
    </html>
  );
}
```

8.2 Dashboard Layout with Sidebar

```
// app/dashboard/layout.js
import Sidebar from '@/components/Sidebar';

export default function DashboardLayout({ children }) {
  return (
    <div style={{ display: 'flex' }}>
      <Sidebar />
      <main style={{ flex: 1 }}>
        {children}
      </main>
    </div>
  );
```

```
}

// All pages in /dashboard get sidebar
```

8.3 Nested Layout with Header

```
// app/admin/layout.js
import AdminHeader from '@/components/AdminHeader';

export default function AdminLayout({ children }) {
  return (
    <>
      <AdminHeader />
      <div className="admin-content">
        {children}
      </div>
    </>
  );
}
```

Pattern: Import custom components in layouts!

Chapter 9

Styling Next.js Projects

9.1 Styling Options

- **CSS Modules** - Component-scoped styles
- **Global CSS** - App-wide styles
- **Tailwind CSS** - Utility classes
- **CSS-in-JS** - Styled Components

9.2 CSS Modules

Create separate `.module.css` file:

```
// components/Button.module.css
.button {
  background-color: #3498db;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.button:hover {
  background-color: #2980b9;
}
```

Use in component:

```
// components/Button.js
import styles from './Button.module.css';

export default function Button() {
  return (
    <button className={styles.button}>
```

```
      Click me
    </button>
  );
}
```

9.3 Global CSS

```
// app/globals.css
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, sans-serif;
  background-color: #f5f5f5;
  color: #333;
}

// app/layout.js
import './globals.css';

export default function Layout({ children }) {
  return <html><body>{children}</body></html>;
}
```

9.4 Tailwind CSS

Already configured in create-next-app:

```
// No imports needed!
export default function Button() {
  return (
    <button className="bg-blue-500 text-white px-4 py-2 rounded">
      Click me
    </button>
  );
}
```

Recommendation: Use CSS Modules or Tailwind!

Chapter 10

Optimizing Images with the Next.js Image Component

10.1 Why Optimize?

- Reduce file size (smaller downloads)
- Serve correct format per device
- Lazy load off-screen images
- Prevent layout shift
- Responsive images

10.2 Using Image Component

```
import Image from 'next/image';

export default function ProfileCard() {
  return (
    <Image
      src="/profile.jpg"
      alt="Profile picture"
      width={400}
      height={300}
      priority  // Load immediately
    />
  );
}
```

10.3 Lazy Loading

```
// Off-screen images load when visible
import Image from 'next/image';
```

```
export default function Gallery() {
  return (
    <div>
      {images.map(img => (
        <Image
          key={img.id}
          src={img.src}
          alt={img.alt}
          width={300}
          height={300}
          // lazy loading automatic unless priority=true
        />
      ))}
    </div>
  );
}
```

10.4 Responsive Images

```
<Image
  src="/hero.jpg"
  alt="Hero"
  width={1200}
  height={400}
  sizes="(max-width: 768px) 100vw,
         (max-width: 1200px) 50vw,
         33vw"
  responsive
/>
```

Always use: Next.js Image component instead of <img>!

Chapter 11

Fetching Data By Leveraging Next.js & Fullstack Capabilities

11.1 Server Components for Data Fetching

```
// app/posts/page.js
async function getPosts() {
  // Runs only on server!
  const response = await fetch('https://api.example.com/posts');
  return response.json();
}

export default async function Posts() {
  const posts = await getPosts();

  return (
    <ul>
      {posts.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

11.2 Database Access

```
// app/users/page.js
import db from '@/lib/database';

export default async function Users() {
  // Direct database access - no API needed!
  const users = await db.user.findMany();

  return (
    <ul>
```

```
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

11.3 API Routes

When you need an API endpoint:

```
// app/api/posts/route.js
export async function GET(request) {
  const posts = await db.post.findMany();
  return Response.json(posts);
}

export async function POST(request) {
  const data = await request.json();
  const post = await db.post.create({ data });
  return Response.json(post);
}

// GET http://localhost:3000/api/posts
// POST http://localhost:3000/api/posts
```

Pattern: Direct data fetching in server components!

Chapter 12

React Server Components vs Client Components

12.1 Server Components (Default)

```
// app/page.js (Server Component)
export default async function Page() {
  const data = await fetch('...');

  return <div>{data}</div>;
}
```

Advantages:

- Direct database access
- API key security
- Smaller bundles
- Server-side rendering

12.2 Client Components

```
// app/components/Counter.js
'use client';  // Mark as client component

import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
```

```
  );
}
```

Use when:

- Need useState, useEffect, hooks
- Need event handlers (onClick, etc.)
- Need browser APIs

12.3 Mixing Both

```
// app/page.js (Server Component)
import Counter from '@/components/Counter';

async function getData() {
  const data = await fetch('...');
  return data;
}

export default async function Page() {
  const data = await getData();

  return (
    <div>
      <p>Data from server: {data.title}</p>
      <Counter />  {/* Client component for interactivity */}
    </div>
  );
}
```

Strategy: Server by default, client for interactivity!

Chapter 13

Adding Loading & Error States

13.1 Loading Page

Show while data loads:

```
// app/loading.js
export default function Loading() {
  return (
    <div className="loading-container">
      <div className="spinner"></div>
      <p>Loading...</p>
    </div>
  );
}
```

This shows for all pages in that folder while loading.

13.2 Error Boundary

Handle errors gracefully:

```
// app/error.js
'use client';

export default function Error({ error, reset }) {
  return (
    <div>
      <h2>Something went wrong!</h2>
      <p>{error.message}</p>
      <button onClick={() => reset()}>
        Try again
      </button>
    </div>
  );
}
```

13.3 Not Found Page

```
// app/not-found.js
export default function NotFound() {
  return (
    <div>
      <h1>404 - Page Not Found</h1>
      <p>The page you're looking for doesn't exist.</p>
    </div>
  );
}
```

Pattern: Special files handle loading/errors automatically!

Chapter 14

Using Suspense & Streamed Responses

14.1 Suspense for Loading States

Show placeholder while data loads:

```
import { Suspense } from 'react';

async function SlowComponent() {
  await new Promise(resolve => setTimeout(resolve, 2000));
  return <div>Loaded!</div>;
}

export default function Page() {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <SlowComponent />
    </Suspense>
  );
}
```

14.2 Multiple Suspense Boundaries

Load different parts at different times:

```
import { Suspense } from 'react';

export default function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>

      <Suspense fallback={<p>Loading user...</p>}>
        <UserCard />
```

```
      </Suspense>

      <Suspense fallback={<p>Loading posts...</p>}>
        <PostsList />
      </Suspense>
    </div>
  );
}
```

Benefit: Users see content as it loads!

Chapter 15

Server Actions for Form Submissions

15.1 What are Server Actions?

Functions that run on the server. Triggered from client.

```
// app/actions.js
'use server';

export async function createPost(formData) {
  const title = formData.get('title');
  const content = formData.get('content');

  const post = await db.post.create({
    data: { title, content }
  });

  return post;
}
```

15.2 Using in Form

```
// app/components/PostForm.js
'use client';

import { createPost } from '@/app/actions';

export default function PostForm() {
  return (
    <form action={createPost}>
      <input name="title" placeholder="Title" required />
      <textarea name="content" placeholder="Content" required />
      <button type="submit">Create Post</button>
    </form>
  );
}
```

15.3 Handling Responses

```
'use client';

import { useState } from 'react';
import { createPost } from '@/app/actions';

export default function PostForm() {
  const [message, setMessage] = useState('');

  async function handleSubmit(e) {
    e.preventDefault();
    const formData = new FormData(e.target);

    try {
      const result = await createPost(formData);
      setMessage('Post created!');
    } catch (error) {
      setMessage('Error: ' + error.message);
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      {/* form fields */}
      {message && <p>{message}</p>}
    </form>
  );
}
```

Magic: Server functions called from client automatically!

Chapter 16

Input Validation & XSS Protection

16.1 Server-Side Validation

Always validate on server:

```
'use server';

export async function createPost(formData) {
  const title = formData.get('title');
  const content = formData.get('content');

  // Validation
  if (!title || title.length < 3) {
    throw new Error('Title must be at least 3 characters');
  }

  if (!content || content.length < 10) {
    throw new Error('Content must be at least 10 characters');
  }

  // Safe to save
  const post = await db.post.create({
    data: { title, content }
  });

  return post;
}
```

16.2 Sanitizing Input

Prevent XSS attacks:

```
import DOMPurify from 'isomorphic-dompurify';

'use server';
```

```
export async function createPost(formData) {
  let title = formData.get('title');
  let content = formData.get('content');

  // Remove malicious HTML
  title = DOMPurify.sanitize(title);
  content = DOMPurify.sanitize(content);

  // Now safe to save
  const post = await db.post.create({
    data: { title, content }
  });

  return post;
}
```

16.3 Creating a Slug

Convert title to URL-safe slug:

```
function createSlug(title) {
  return title
    .toLowerCase()
    .trim()
    .replace(/[^\w\s-]/g, '')  // Remove special chars
    .replace(/[\s_-]+/g, '-')  // Replace spaces/underscores with dash
    .replace(/^-+|-+$/g, '');  // Remove leading/trailing dashes
}

// "Hello World!" → "hello-world"
// "My-Post (2024)" → "my-post-2024"
```

Security: Always validate and sanitize server-side!

Chapter 17

Storing Images & Data in Database

17.1 File Upload

```
'use client';

import { uploadImage } from '@/app/actions';

export default function ImageUploadForm() {
  async function handleSubmit(e) {
    e.preventDefault();
    const formData = new FormData(e.target);

    const result = await uploadImage(formData);
    console.log('Uploaded:', result);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="image" type="file" accept="image/*" required />
      <button type="submit">Upload</button>
    </form>
  );
}
```

17.2 Server-Side Upload

```
'use server';

import { writeFile } from 'fs/promises';
import { join } from 'path';

export async function uploadImage(formData) {
  const file = formData.get('image');

  if (!file) throw new Error('No file provided');
```

```
  // Read file
  const bytes = await file.arrayBuffer();
  const buffer = Buffer.from(bytes);

  // Save to public folder
  const filename = \`\${Date.now()}-\${file.name}\`;
  const path = join(process.cwd(), 'public/uploads', filename);

  await writeFile(path, buffer);

  return { url: \`/uploads/\${filename}\` };
}
```

17.3 Storing in Database

```
'use server';

export async function createPostWithImage(formData) {
  const title = formData.get('title');
  const file = formData.get('image');

  // Upload image
  const { url } = await uploadImage(formData);

  // Save post with image URL
  const post = await db.post.create({
    data: {
      title,
      imageUrl: url
    }
  });

  return post;
}
```

Note: Use external services for production!

Chapter 18

Building For Production & Caching

18.1 Build Command

`npm run build`

Creates optimized production build.

18.2 Understanding Caching

Next.js caches:

- Server-side rendering results
- Static pages
- API responses
- Images

18.3 Revalidating Cache

Tell Next.js to refresh cached data:

```
// app/blog/page.js
import { revalidatePath } from 'next/cache';

async function getBlogs() {
  const blogs = await fetch('https://api.example.com/blogs', {
    next: { revalidate: 60 }  // Revalidate every 60 seconds
  });
  return blogs.json();
}

export default async function Blog() {
  const blogs = await getBlogs();
  return <div>{/* render blogs */}</div>;
}
```

18.4 On-Demand Revalidation

```
'use server';

import { revalidatePath } from 'next/cache';

export async function createPost(formData) {
  const post = await db.post.create({
    data: { /* ... */ }
  });

  // Refresh the blog page
  revalidatePath('/blog');

  return post;
}
```

Pattern: Revalidate after data changes!

Chapter 19

Adding Metadata

19.1 Static Metadata

```
// app/layout.js
export const metadata = {
  title: 'My Blog',
  description: 'Read my latest posts',
  openGraph: {
    title: 'My Blog',
    description: 'Read my latest posts',
    url: 'https://myblog.com',
    siteName: 'My Blog',
    images: [
      {
        url: 'https://myblog.com/og-image.jpg',
        width: 1200,
        height: 630
      }
    ]
  }
};
```

19.2 Dynamic Metadata

```
// app/blog/[slug]/page.js
export async function generateMetadata({ params }) {
  const post = await db.post.findUnique({
    where: { slug: params.slug }
  });

  return {
    title: post.title,
    description: post.excerpt,
    openGraph: {
      title: post.title,
```

```
      description: post.excerpt,
      images: [{ url: post.image }]
    }
  };
}

export default async function PostPage({ params }) {
  const post = await db.post.findUnique({
    where: { slug: params.slug }
  });

  return <article>{post.content}</article>;
}
```

SEO: Metadata helps search engines and social sharing!

Part II

Next.js Pages Router (Legacy Approach)

Chapter 20

Introduction to Pages Router

20.1 Pages Router vs App Router

Feature	Pages Router	App Router
Introduced	Early Next.js	Next.js 13+
Status	Stable, works	Modern, recommended
Syntax	Simple	More powerful
Learning curve	Easy	Medium

Pages Router still works and many projects use it.

20.2 File Structure

```
pages/
  index.js           → /
  about.js           → /about
  blog.js            → /blog
  blog/
     [slug].js       → /blog/my-post
  api/
     posts.js        → /api/posts

public/
api/
styles/
```

20.3 Creating First Page

```
// pages/index.js
export default function Home() {
  return <h1>Welcome</h1>;
}

// Automatically becomes /
```

Chapter 21

Pages Router: Adding Routes & Dynamic Pages

21.1 Basic Routes

```
pages/about.js      → /about
pages/contact.js    → /contact
pages/products.js   → /products
```

21.2 Nested Routes

```
pages/blog/index.js      → /blog
pages/blog/[slug].js     → /blog/my-post
pages/blog/archive.js    → /blog/archive
```

21.3 Dynamic Pages

```
// pages/products/[id].js
import { useRouter } from 'next/router';

export default function Product() {
  const router = useRouter();
  const { id } = router.query;

  return <h1>Product {id}</h1>;
}

// /products/123 → id = '123'
```

21.4 Catch-all Routes

```
// pages/docs/[...slug].js
export default function Docs() {
  const router = useRouter();
```

```
  const { slug } = router.query;

  // /docs/guide/intro → slug = ['guide', 'intro']
}
```

Chapter 22

Pages Router: Data Fetching

22.1 Static Generation (SSG)

```
// pages/posts/index.js
function Posts({ posts }) {
  return (
    <ul>
      {posts.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}

export async function getStaticProps() {
  const posts = await fetch('/api/posts').then(r => r.json());

  return {
    props: { posts },
    revalidate: 3600  // Revalidate every hour
  };
}

export default Posts;
```

22.2 Server-Side Rendering (SSR)

```
// pages/posts/index.js
function Posts({ posts }) {
  return <ul>{/* render posts */}</ul>;
}

export async function getServerSideProps() {
  // Runs on every request
  const posts = await fetch('/api/posts').then(r => r.json());
```

```
  return {
    props: { posts }
  };
}

export default Posts;
```

22.3 Dynamic Pages with getStaticPaths

```
// pages/posts/[slug].js
function Post({ post }) {
  return <article>{post.content}</article>;
}

export async function getStaticPaths() {
  const posts = await fetch('/api/posts').then(r => r.json());

  const paths = posts.map(post => ({
    params: { slug: post.slug }
  }));

  return {
    paths,
    fallback: 'blocking'  // Generate missing pages on demand
  };
}

export async function getStaticProps({ params }) {
  const post = await fetch(\`/api/posts/\${params.slug}\`)
    .then(r => r.json());

  return {
    props: { post },
    revalidate: 3600
  };
}

export default Post;
```

Chapter 23

Pages Router: $_{a}pp.js \& Layouts$

23.1 The $_{a}pp.jsFile$

Global wrapper for all pages:

```
// pages/_app.js
import '../styles/globals.css';

function MyApp({ Component, pageProps }) {
  return (
    <div>
      <header>Header</header>
      <Component {...pageProps} />
      <footer>Footer</footer>
    </div>
  );
}

export default MyApp;
```

23.2 With Providers

```
// pages/_app.js
import { QueryClientProvider, QueryClient } from '@tanstack/react-query';

const queryClient = new QueryClient();

function MyApp({ Component, pageProps }) {
  return (
    <QueryClientProvider client={queryClient}>
      <Component {...pageProps} />
    </QueryClientProvider>
  );
}
```

```
export default MyApp;
```

Chapter 24

Pages Router: Navigation & Linking

24.1 Link Component

```
import Link from 'next/link';

export default function Home() {
  return (
    <nav>
      <Link href="/">Home</Link>
      <Link href="/about">About</Link>
      <Link href={`/blog/${postId}`}>Blog</Link>
    </nav>
  );
}
```

24.2 useRouter for Navigation

```
import { useRouter } from 'next/router';

export default function Button() {
  const router = useRouter();

  const handleClick = () => {
    router.push('/dashboard');
  };

  return <button onClick={handleClick}>Go</button>;
}
```

Chapter 25

Pages Router: API Routes

25.1 Creating API Routes

```
// pages/api/posts.js
export default function handler(req, res) {
  if (req.method === 'GET') {
    const posts = getPosts();
    res.status(200).json(posts);
  } else if (req.method === 'POST') {
    const post = createPost(req.body);
    res.status(201).json(post);
  }
}

// GET http://localhost:3000/api/posts
// POST http://localhost:3000/api/posts
```

25.2 Dynamic API Routes

```
// pages/api/posts/[id].js
export default function handler(req, res) {
  const { id } = req.query;

  if (req.method === 'GET') {
    const post = getPost(id);
    res.json(post);
  } else if (req.method === 'PUT') {
    const post = updatePost(id, req.body);
    res.json(post);
  } else if (req.method === 'DELETE') {
    deletePost(id);
    res.status(204).end();
  }
}
```

```
// GET /api/posts/1
// PUT /api/posts/1
// DELETE /api/posts/1
```

Chapter 26

Pages Router: Deployment

26.1 Build and Deploy

```
npm run build
npm run start
```

26.2 Vercel Deployment

1. Push code to GitHub
2. Go to vercel.com
3. Connect repository
4. Deploy
5. Automatic deployments on push

26.3 Fallback Pages

Show while regenerating:

```
export async function getStaticPaths() {
  return {
    paths: [],  // No pregenerated pages
    fallback: 'blocking'  // Generate on demand
  };
}
```

Or show loading page:

```
fallback: true  // Show loading, then page
```

Quick Summary

App Router Essentials

1. Files in `app/` = routes
2. `page.js` = route content
3. `layout.js` = shared wrapper
4. `[id]` = dynamic segment
5. Server components by default
6. Suspense for loading states
7. Server Actions for forms

Pages Router Essentials

1. Files in `pages/` = routes
2. getStaticProps = static generation
3. getServerSideProps = server rendering
4. getStaticPaths = dynamic page paths
5. $_{a}app.js = global wrapper API routes in$ `pages/api/`

When to Use Each

Use App Router if:

6. Building new project
- Want server components
- Want modern features

Use Pages Router if:

- Existing project uses it
- Need legacy support
- Familiar with old approach

Key Concepts

- File-based routing
- Server components for data fetching
- Client components for interactivity
- Server Actions for forms
- Image optimization
- API routes
- Metadata for SEO

Next.js makes building full-stack React apps simple and fast!